The Journal of Open Source Software

NeuroCAPs: A Python Package for Performing Co-Activation Patterns Analyses on Resting-State and Task-Based fMRI Data

Donisha Smith[1]

1 Department of Psychology, Florida International University

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: julia ferraioli

Reviewers:

- @robbisg
- @yibeichan

Submitted: 21 March 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Co-Activation Patterns (CAPs) is a dynamic functional connectivity technique that clusters similar spatial distributions of brain activity. To make this analytical technique more accessible to neuroimaging researchers, NeuroCAPs, an open source Python package, was developed. This package performs end-to-end CAPs analyses on preprocessed resting-state or task-based functional magnetic resonance imaging (fMRI) data, and is most optimized for data preprocessed with fMRIPrep, a robust preprocessing pipeline designed to minimize manual user input and enhance reproducibility (Esteban et al., 2019).

Background

Numerous fMRI studies employ static functional connectivity (sFC) techniques to analyze correlative activity within and between brain regions. However, these approaches operate under the assumption that functional connectivity patterns, which change within seconds (Jiang et al., 2022), remain stationary throughout the entire data acquisition period (Hutchison et al., 2013).

Unlike sFC approaches, dynamic functional connectivity (dFC) methods enable the analysis of dynamic functional states, which are characterized by consistent, replicable, and distinct periods of time-varying brain connectivity patterns (Rabany et al., 2019). Among these techniques, CAPs analysis aggregates similar spatial distributions of brain activity using clustering techniques, typically the k-means algorithm, to capture the dynamic nature of brain activity (Liu et al., 2013, 2018).

Statement of Need

The typical CAPs workflow can be programmatically time-consuming to manually orchestrate as it generally entails several steps:

1. implement spatial dimensionality reduction of timeseries data
2. perform nuisance regression and scrub high-motion volumes (excessive head motion)
3. concatenate the timeseries data from multiple subjects into a single matrix
4. apply k-means clustering to the concatenated data and select the optimal number of clusters (CAPs) using heuristics such as the elbow or silhouette methods
5. generate different visualizations to enhance the interpretability of the CAP

While other excellent CAPs toolboxes exist, they are often implemented in proprietary languages such as MATLAB (which is the case for TbCAPs (Bolton et al., 2020)), lack comprehensive end-to-end analytical pipelines for both resting-state and task-based fMRI data with temporal

dynamic metrics and visualization capabilities (such as capcalc (Frederick & Drucker, 2022)), or are comprehensive, but generalized toolboxes for evaluating and comparing different dFC methods (such as pydFC (Torabi et al., 2024)).

NeuroCAPs addresses these limitations by providing an accessible Python package specifically for performing end-to-end CAPs analyses, from post-processing of fMRI data to creation of temporal metrics for downstream statistical analyses and visualizations to facilitate interpretations. However, many of NeuroCAPs' post-processing functionalities assumes that fMRI data is organized in a Brain Imaging Data Structure (BIDS) compliant directory and is most optimized for data preprocessed with fMRIPrep (Esteban et al., 2019) or preprocessing pipelines that generate similar outputs (e.g. NiBabies (Goncalves et al., 2025)). Furthermore, NeuroCAPs only supports the k-means algorithm for clustering, which is the clustering algorithm that was originally used and is often employed when performing the CAPs analysis (Liu et al., 2013).

Modules

The core functionalities of NeuroCAPs are concentrated in three modules:

1. `neurocaps.extraction` Contains the `TimeseriesExtractor` class, which:

   - collects preproccessed BOLD data from an BIDS-compliant dataset (Yarkoni et al., 2019)
   - leverages Nilearn's (contributors, n.d.) `NiftiLabelsMasker` to perform nuisance regression and spatial dimensionality reduction using deterministic parcellations (e.g., Schaefer (Schaefer et al., 2018), AAL (Tzourio-Mazoyer et al., 2002))
   - scrubs high-motion volumes using fMRIPrep-derived framewise displacement values
   - reports quality control information related to high-motion or non-steady state volumes

2. `neurocaps.analysis` Contains the CAP class for performing the main analysis, as well as several standalone utility functions.

   - The `CAP` class:
     - performs k-means clustering (Pedregosa et al., 2011) to identify CAPs, supporting both single and optimized cluster selection with heuristics such as the silhouette and elbow method (Arvai, 2023)
     - computes subject-level temporal dynamics metrics (e.g., fractional occupancy, transition probabilities) for statistical analysis
     - converts identified CAPs back into NIfTI statistical maps for spatial interpretation
     - integrates multiple plotting libraries (Gale et al., 2021; Hunter, 2007; Inc., n.d.; Waskom, 2021) to provide a diverse range of visualization options
   - Standalone functions: Provide tools for data standardization (Harris et al., 2020), merging timeseries data across sessions or tasks, and creating group-averaged transition matrices.

3. `neurocaps.utils`

Contains several utility functions:

- `fetch_preset_parcel_approach`: fetches a preset parcel approach (either "4S" or "HCPex" (Huang et al., 2022))
- `generate_custom_parcel_approach`: automatically creates the necessary data structures from a parcellation's metadata file

Additional utility function are also available for plotting and simulating data.

Workflow Example

The following code demonstrates a simple workflow example using NeuroCAPs to perform the CAPs analysis. Note that this example uses simulated data, an interactive variant of a workflow example using real data is available on the readthedocs.

DRAFT

1. Extract timeseries data

```python
import numpy as np

from neurocaps.extraction import TimeseriesExtractor
from neurocaps.utils import simulate_bids_dataset

# Set seed
np.random.seed(0)

# Generate a BIDS directory with fMRIPrep derivatives
bids_root = simulate_bids_dataset(n_subs=3, n_runs=1, n_volumes=100, task_name="rest")

# Using Schaefer, one of the default parcellation approaches
parcel_approach = {"Schaefer": {"n_rois": 100, "yeo_networks": 7}}

# List of fMRIPrep-derived confounds for nuisance regression
confound_names = [
    "cosine*",
    "trans*",
    "rot*",
    "a_comp_cor_00",
    "a_comp_cor_01",
    "a_comp_cor_02",
    "a_comp_cor_03",
    "a_comp_cor_04",
]

# Initialize extractor with signal cleaning parameters
extractor = TimeseriesExtractor(
    space="MNI152NLin2009cAsym",
    parcel_approach=parcel_approach,
    confound_names=confound_names,
    standardize=False,
    fd_threshold={
        "threshold": 0.50,
        "outlier_percentage": 0.30,
    },
)

# Extract BOLD data from preprocessed fMRIPrep data
# which should be located in the "derivatives" folder
# within the BIDS root directory
# The extracted timeseries data is automatically stored
extractor.get_bold(bids_dir=bids_root, task="rest", tr=2, n_cores=1, verbose=False)

# Retrieve the dataframe containing QC information for each subject
# to use for downstream statistical analyses
qc_df = extractor.report_qc()
print(qc_df)
```

DRAFT

	Subject_ID	Run	Mean_FD	Std_FD	Frames_Scrubbed	Frames_Interpolated	Mean_High_Motion_Length	Std_High_Motion_Length	N_Dummy_Scans
0	0	run-0	0.516349	0.289657	9	0	1.125000	0.330719	NaN
1	1	run-0	0.526343	0.297550	17	0	1.133333	0.339935	NaN
2	2	run-0	0.518041	0.273964	8	0	1.000000	0.000000	NaN

Figure 1: Quality Control Dataframe.

2. Use k-means clustering to identify the optimal number of CAPs from the data using a heuristic

```
from neurocaps.analysis import CAP

# Initialize CAP class
cap_analysis = CAP(parcel_approach=extractor.parcel_approach)

# Identify the optimal number of CAPs (clusters)
# using the silhouette method to test 2-20
# The optimal number of CAPs is automatically stored
cap_analysis.get_caps(
    subject_timeseries=extractor.subject_timeseries,
    n_clusters=range(2, 21),
    standardize=True,
    cluster_selection_method="silhouette",
    max_iter=500,
    n_init=10,
)
```

3. Compute temporal dynamic metrics for downstream statistical analyses

```
# Calculate temporal fraction of each CAP for all subjects
output = cap_analysis.calculate_metrics(extractor.subject_timeseries, metrics=["temporal
print(output["temporal_fraction"])
```

	Subject_ID	Group	Run	CAP-1	CAP-2
0	0	All Subjects	run-0	0.571429	0.428571
1	1	All Subjects	run-0	0.602410	0.397590
2	2	All Subjects	run-0	0.652174	0.347826

Figure 2: Temporal Fraction Dataframe.

4. Visualize CAPs

```
# Project CAPs onto surface plots
# and generate cosine similarity network alignment of CAPs
from neurocaps.utils import PlotDefaults

surface_kwargs = PlotDefaults.caps2surf()
surface_kwargs["layout"] = "row"
surface_kwargs["size"] = (500, 100)

radar_kwargs = PlotDefaults.caps2radar()
radar_kwargs["height"] = 400
radar_kwargs["width"] = 600

radialaxis = {
    "showline": True,
```

DRAFT

```
    "linewidth": 2,
    "linecolor": "rgba(0, 0, 0, 0.25)",
    "gridcolor": "rgba(0, 0, 0, 0.25)",
    "ticks": "outside",
    "tickfont": {"size": 14, "color": "black"},
    "range": [0, 0.4],
    "tickvals": [0.1, "", "", 0.4],
}

legend = {
    "yanchor": "top",
    "y": 0.99,
    "x": 0.99,
    "title_font_family": "Times New Roman",
    "font": {"size": 12, "color": "black"},
}

radar_kwargs["radialaxis"] = radialaxis
radar_kwargs["legend"] = legend

cap_analysis.caps2surf(**surface_kwargs).caps2radar(**radar_kwargs)
```

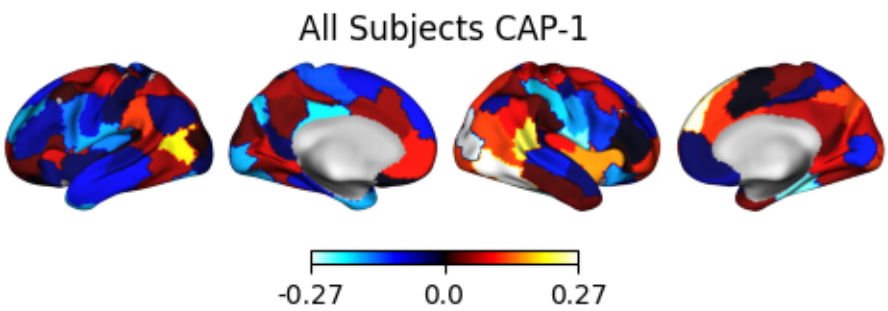

Figure 3: CAP-1 Surface Image.

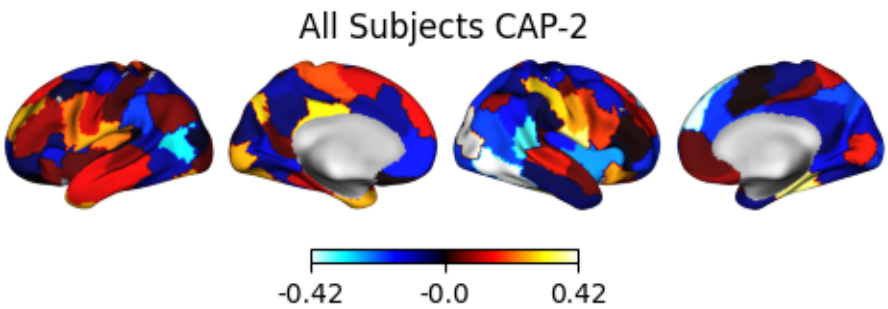

Figure 4: CAP-2 Surface Image.

DRAFT

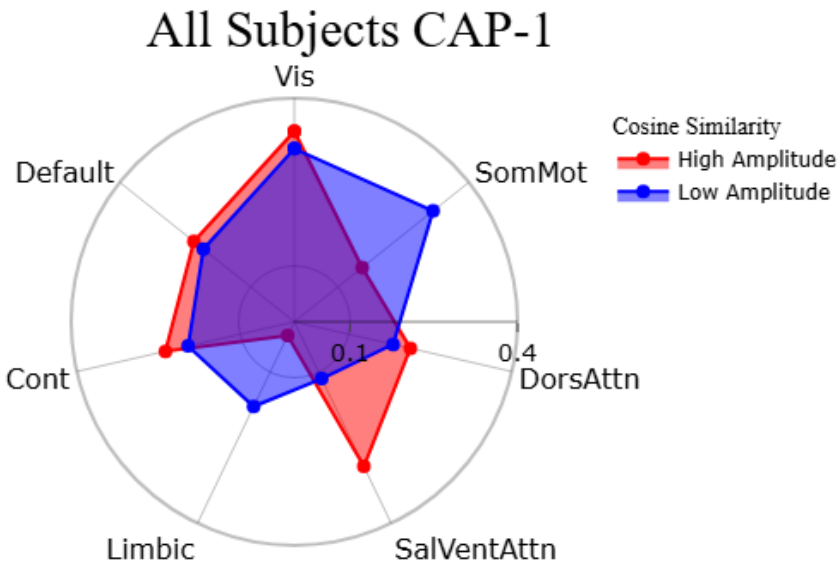

Figure 5: CAP-1 Radar Image.

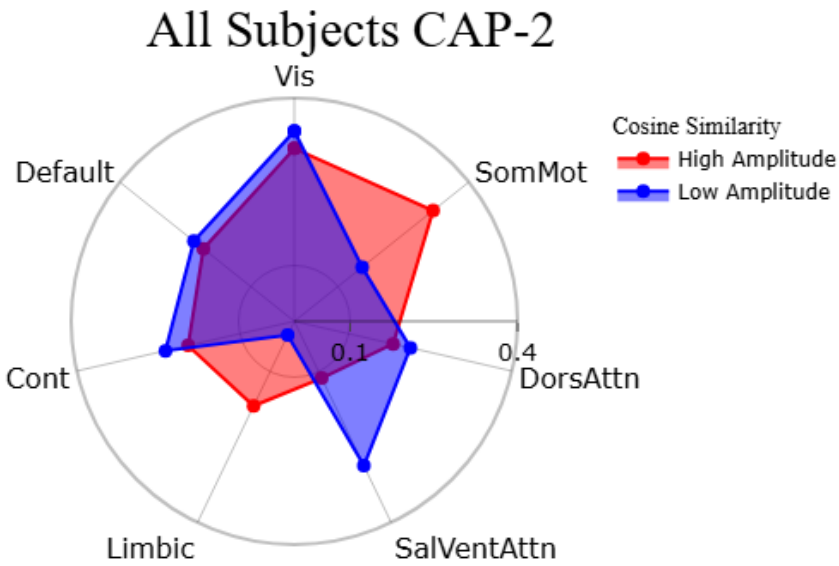

Figure 6: CAP-2 Radar Image.

Documentation

Comprehensive documentations and interactive tutorials of NeuroCAPS can be found at https://neurocaps.readthedocs.io/ and on its repository.

Research Utility

NeuroCAPs was originally developed (and later expanded and refined for broader use) to facilitate the analysis in Smith et al. (2025), which has been submitted for peer review by the same author.

Acknowledgements

Funding provided by the Dissertation Year Fellowship (DYF) Program at Florida International University (FIU) assisted in further refinement and expansion of NeuroCAPs.

References

Arvai, K. (2023). *Kneed*. Zenodo. https://doi.org/10.5281/ZENODO.8127224

Bolton, T. A. W., Tuleasca, C., Wotruba, D., Rey, G., Dhanis, H., Gauthier, B., Delavari, F., Morgenroth, E., Gaviria, J., Blondiaux, E., Smigielski, L., & Van De Ville, D. (2020). TbCAPs: A toolbox for co-activation pattern analysis. *NeuroImage*, *211*, 116621. https://doi.org/10.1016/j.neuroimage.2020.116621

contributors, N. (n.d.). *nilearn*. https://doi.org/10.5281/zenodo.8397156

Esteban, O., Markiewicz, C. J., Blair, R. W., Moodie, C. A., Isik, A. I., Erramuzpe, A., Kent, J. D., Goncalves, M., DuPre, E., Snyder, M., Oya, H., Ghosh, S. S., Wright, J., Durnez, J., Poldrack, R. A., & Gorgolewski, K. J. (2019). fMRIPrep: A robust preprocessing pipeline for functional MRI. *Nature Methods*, *16*(1), 111–116. https://doi.org/10.1038/s41592-018-0235-4

Frederick, B. D., & Drucker, D. M. (2022). *Bbfrederick/capcalc: Version 1.2.2.2 - 8/30/22 deployment bug fix*. Zenodo. https://doi.org/10.5281/ZENODO.7035806

Gale, D. J., Vos de Wael., R., Benkarim, O., & Bernhardt, B. (2021). *Surfplot: Publication-ready brain surface figures*. Zenodo. https://doi.org/10.5281/ZENODO.5567926

Goncalves, M., Markiewicz, C. J., Esteban, O., Feczko, E., Poldrack, R. A., & Fair, D. A. (2025). *NiBabies: A robust preprocessing pipeline for infant functional MRI*. Zenodo. https://doi.org/10.5281/ZENODO.14811979

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Huang, C.-C., Rolls, E. T., Feng, J., & Lin, C.-P. (2022). An extended human connectome project multimodal parcellation atlas of the human cortex and subcortical areas. *Brain Structure and Function*, *227*(3), 763–778. https://doi.org/10.1007/s00429-021-02421-6

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Hutchison, R. M., Womelsdorf, T., Allen, E. A., Bandettini, P. A., Calhoun, V. D., Corbetta, M., Della Penna, S., Duyn, J. H., Glover, G. H., Gonzalez-Castillo, J., Handwerker, D. A., Keilholz, S., Kiviniemi, V., Leopold, D. A., De Pasquale, F., Sporns, O., Walter, M., & Chang, C. (2013). Dynamic functional connectivity: Promise, issues, and interpretations. *NeuroImage*, *80*, 360–378. https://doi.org/10.1016/j.neuroimage.2013.05.079

Inc., P. T. (n.d.). *Chart title*. https://plotly.com/python/radar-chart/; Plotly Technologies Inc.

DRAFT

Jiang, F., Jin, H., Gao, Y., Xie, X., Cummings, J., Raj, A., & Nagarajan, S. (2022). Time-varying dynamic network model for dynamic resting state functional connectivity in fMRI and MEG imaging. *NeuroImage*, *254*, 119131. https://doi.org/10.1016/j.neuroimage.2022.119131

Liu, X., Chang, C., & Duyn, J. H. (2013). Decomposition of spontaneous brain activity into distinct fMRI co-activation patterns. *Frontiers in Systems Neuroscience*, *7*. https://doi.org/10.3389/fnsys.2013.00101

Liu, X., Zhang, N., Chang, C., & Duyn, J. H. (2018). Co-activation patterns in resting-state fMRI signals. *NeuroImage*, *180*, 485–494. https://doi.org/10.1016/j.neuroimage.2018.01.041

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Rabany, L., Brocke, S., Calhoun, V. D., Pittman, B., Corbera, S., Wexler, B. E., Bell, M. D., Pelphrey, K., Pearlson, G. D., & Assaf, M. (2019). Dynamic functional connectivity in schizophrenia and autism spectrum disorder: Convergence, divergence and classification. *NeuroImage: Clinical*, *24*, 101966. https://doi.org/10.1016/j.nicl.2019.101966

Schaefer, A., Kong, R., Gordon, E. M., Laumann, T. O., Zuo, X.-N., Holmes, A. J., Eickhoff, S. B., & Yeo, B. T. T. (2018). Local-global parcellation of the human cerebral cortex from intrinsic functional connectivity MRI. *Cerebral Cortex*, *28*(9), 3095–3114. https://doi.org/10.1093/cercor/bhx179

Smith, D. D., Bartley, J. E., Peraza, J. A., Bottenhorn, K. L., Nomi, J. S., Uddin, L. Q., Riedel, M. C., Salo, T., Laird, R. W., Pruden, S. M., Sutherland, M. T., Brewe, E., & Laird, A. R. (2025). *Dynamic reconfiguration of brain coactivation states associated with active and lecture-based learning of university physics*. https://doi.org/10.1101/2025.02.22.639361

Torabi, M., Mitsis, G. D., & Poline, J.-B. (2024). On the variability of dynamic functional connectivity assessment methods. *GigaScience*, *13*, giae009. https://doi.org/10.1093/gigascience/giae009

Tzourio-Mazoyer, N., Landeau, B., Papathanassiou, D., Crivello, F., Etard, O., Delcroix, N., Mazoyer, B., & Joliot, M. (2002). Automated anatomical labeling of activations in SPM using a macroscopic anatomical parcellation of the MNI MRI single-subject brain. *NeuroImage*, *15*(1), 273–289. https://doi.org/10.1006/nimg.2001.0978

Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, *6*(60), 3021. https://doi.org/10.21105/joss.03021

Yarkoni, T., Markiewicz, C., De La Vega, A., Gorgolewski, K., Salo, T., Halchenko, Y., McNamara, Q., DeStasio, K., Poline, J.-B., Petrov, D., Hayot-Sasson, V., Nielson, D., Carlin, J., Kiar, G., Whitaker, K., DuPre, E., Wagner, A., Tirrell, L., Jas, M., … Blair, R. (2019). PyBIDS: Python tools for BIDS datasets. *Journal of Open Source Software*, *4*(40), 1294. https://doi.org/10.21105/joss.01294

DRAFT